

Jared Esquivel

Final Project Proposal

My final project proposal is an Animal Crossing social media website that will allow the user to see the villagers seen across the Animal Crossing games talk to each other (I will not do every single one, as that will require too much dialogue), and will respond if the user posts something. Regardless of whether you are logged in or not, you can see the messages, but to send a message, you must be logged into an account. In the header, there will be a create account page and a log in page so that you may make an account to post stuff or to log in to continue posting things. The villagers will be sending messages every 40 - 60 seconds and will consist of messages from 1 to 3 villagers, but if you post something, they will post a message from 5 to 15 seconds after you make a post. Now, to get into the details, let's start with the most important part of the backend models. This project will require 3 models. The first model is the User model, which will be from `django.contrib.auth.models`. This is because the authentication that is built into Django will allow us to create a good, secure login portal to protect the users of the website. The second model will be the names of all the villagers that will be sending messages, and this will use the string data type. This model exists to randomly select a villager from this model to make N posts. In addition to just being used for the random selection of a villager, it will serve the purpose of being the primary key to the messages model, so that on the deletion of a village, it will delete all messages in cascade, as we don't want to reference a key that no longer exists. Some of the other columns will be (Gender: String), (Personality, String), (Species, String), (Birthday, DateField), (Initial Catchphrase, String), (Skill, String), and (Goal, String). Next is the messages model, which will contain all the messages of the villagers. The data columns will be the villager name as a foreign key to the names model and will be of data type string. Then the second column will

be messages that the villagers can post using a TextField as the data type. Last but not least is the posts model, which will contain all the posts being made by the villagers and users and will have the column name with data type string and message with the data type TextField. The first thing you may think is that this is similar to the villager messages model, but the reason is to isolate all posts from the predefined messages and to be able to delete all the posts in case there are too many to display on the screen. This model can end up being the largest use of data of all the models, as it will need to use lots of data using a TextField to hold all the messages being posted.

The next part of the website will be the front end, and let's start off with the header that will have four options on the left side of the header, and they will be in the order of Home, Posts, Create custom message, and lastly a Don't Sue. Next, on the right side of the header, we will have a Create account and login page. For the home page, we will have a short description with a picture of some Animal Crossing villagers at the top, and if you scroll down, it will show all the villagers that can make posts and show information about each one taken from their original game data. Second is the posts page. This page will always be available and will show all posts being made by the user and villagers, with an option on the right-hand side to create a post using a form to take textfield data and post it to the posts model; this form will only be available if the user is logged in. One last feature of this page will be a delete all posts button in case the user wants to clean up the page because there are too many posts to scroll down from. The next page is for the user to make custom messages for the villagers it will just be a form with a dropdown at the top showing all the villagers so that the user may pick one and then there will be a text field for you to write your custom message and last there will be a submit button to submit your new message to the villager messages model (These messages won't be easily deletable without

having to manually go in and delete them from the models data as a warning). This page will also only be available if there is a user logged into the page. Next is the Don't Sue page, and it will only have a text in the middle saying that this website is for a project and for Nintendo not to sue me for using Animal Crossing characters (I can remove this page, but I just thought that it would be funny for this assignment). Moving on the right hand side of the header we have the create account page which will be a form asking the user for their first name, last name, their preferred email address, and last their password that they would like to use this page is fancy but something just to make an account but this page will be hidden once the user is logged in and will be replaced link to log out of the website. Lastly, the login page, which will request the user to input their email address and a password to log in, will allow them to create custom messages and create posts. I am still thinking of what color scheme to use for the website, but it will most likely be along the lines of bright, vibrant colors, as that is what feels best for the color and interesting design of the Animal Crossing villagers.

Class assignment projects summary

(Github link to all projects done in class: https://github.com/S1ar1dust/CS3620_Portfolio.git)

Personal Portfolio App: This project was fine for the beginning, consisting of us learning the basics of setting up a Django project, creating pages using templates, creating and using models to input and display data to our website, and making a login page with authentication. Personally, I didn't like this part of the project, as I needed to search for almost everything, as this was our first Django project, and it just made it really hard to find a starting point for some of the complete parts, like displaying the data from the models into the HTML, and login authentication is just very confusing to understand.

Book REST API App: This app was about us creating a book library using the rest framework at first to show an easy, faster way to implement the data from the model into Viewsets. Then we learn to display this data on a view through the use of pagination so that we can have X amount of rows in a model per page. Lastly, we used filtering to be able to run "Like" query commands on our model to retrieve specific rows instead of always pulling all the data. This overall was my favorite part of class, as it was fun to do, and at least now I have an understanding of how Django works, and it smoothed out the process when creating this project.